

Delivery 2: F3 - Hyperbolic Sine $\sinh(x)$

From-scratch Python implementation with Tkinter GUI

Arvind Lakshmanan (40310757)

SOEN 6011: Software Engineering Processes
Concordia University

Summer 2026

Assigned function: **F3:** $\sinh(x)$

D2 Roadmap

- ➊ Modify D1 implementation into a from-scratch version.
- ➋ Add a Tkinter graphical user interface.
- ➌ Handle exceptions with helpful messages.
- ➍ Prepare public version-control evidence.
- ➎ Update requirements based on the D2 implementation.

D1 to D2 Change

D1 implementation

- Textual interface.
- Used direct formula:

$$\sinh(x) = \frac{e^x - e^{-x}}{2}$$

- Used `math.exp`.

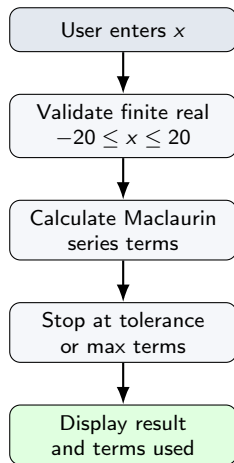
D2 implementation

- Tkinter GUI.
- Uses Maclaurin series:

$$\sinh(x) = x + \frac{x^3}{3!} + \frac{x^5}{5!} + \dots$$

- No Python math-library function for calculation.

Problem 5: From-scratch Algorithm



The implementation separates validation, calculation, stopping logic, and output.

Problem 5: Series Recurrence

```
term_0 = x
sum = x

term_n = term_(n-1) * x^2 / ((2n) * (2n + 1))
sum = sum + term_n

stop when:
|term_n| <= tolerance * (1 + |sum|)
```

Scratch implementation evidence

The calculation avoids `math.sinh`, `math.exp`, `math.factorial`, NumPy, and SciPy.

Problem 5: GUI and Exceptions

Tkinter GUI includes

- input box for x ,
- Calculate button,
- Clear button,
- Exit button,
- result and status area.

Handled errors

- blank input,
- non-numeric input,
- NaN or infinity,
- out-of-range input,
- convergence failure.

Problem 5: Tkinter Application Initialisation

Main GUI Class

The `SinhCalculatorApp` class contains the graphical user interface and connects the interface to the $\sinh(x)$ calculation.

`__init__(self, root)` Initialises the application and receives the main Tkinter window as the `root` parameter.

`root.title()` Sets the title displayed at the top of the application window.

`root.geometry()` Defines the initial window size as 640×420 pixels.

`root.minsize()` Prevents the user from resizing the window below 560×360 pixels.

`tk.StringVar()` Creates variables that automatically connect Python values to Tkinter widgets.

`self.input_value` Stores the value entered by the user in the input field.

`self.status_value` Stores the status message displayed at the bottom of the GUI.

`self.create_widgets()` Calls the method responsible for creating and arranging all interface components.

Problem 5: Tkinter Widgets Used in the Application

<code>ttk.Frame</code>	Groups related widgets into sections such as the main area, input area, and button area.
<code>ttk.Label</code>	Displays the application title, instructions, input label, result label, and status message.
<code>ttk.Entry</code>	Provides the text field where the user enters the value of x . It is connected to <code>self.input_value</code> .
<code>ttk.Button</code>	Creates the Calculate, Clear, and Exit buttons. Each button calls a specific method through its <code>command</code> property.
<code>tk.Text</code>	Displays the calculated result, number of terms used, algorithm description, or error message.
<code>pack()</code>	Places widgets inside their parent frames and controls their position, spacing, and resizing behaviour.
<code>fill="x"</code>	Allows a widget or frame to expand horizontally.
<code>fill="both"</code>	Allows the result area to expand horizontally and vertically.
<code>expand=True</code>	Uses the available extra space when the window is resized.
<code>padx</code> and <code>pady</code>	Add horizontal and vertical spacing between interface components.

Problem 5: Tkinter Event Handling and GUI Functions

- `input_entry.focus()` Places the cursor inside the input field when the program starts, allowing the user to type immediately.
- `input_entry.bind("<Return>", self.calculate)` Connects the Enter key to the `calculate()` method. Therefore, the user can calculate the result without clicking the button.
- `command=self.calculate` Calls the calculation method when the Calculate button is clicked.
- `command=self.clear` Calls the `clear()` method when the Clear button is clicked.
- `command=self.root.destroy` Closes the Tkinter window when the Exit button is clicked.
- `calculate(self, event=None)` Reads the input, validates it, calls the from-scratch $\sinh(x)$ algorithm, and displays either the result or an error.
- `set_result_text(self, message)` Temporarily enables the result box, removes its previous content, inserts the new message, and disables editing again.
 - `clear(self)` Clears the input field, resets the result box, and restores the initial status message.
- `status_value.set()` Changes the status label after a successful calculation, invalid input, convergence failure, or unexpected error.

Problem 5: Starting the Tkinter Application

The main() Function

The `main()` function creates the root window, creates an instance of the GUI class, and starts Tkinter's event loop.

❶ `root = tk.Tk()`

Creates the main application window.

❷ `app = SinhCalculatorApp(root)`

Creates the calculator interface inside the root window.

❸ `root.mainloop()`

Starts Tkinter's event loop and keeps the application running while waiting for user actions.

Problem 5: Manual Validation Evidence

Input	Approximate result	Case
0	0	zero
1	1.1752011936438	positive
-1	-1.1752011936438	negative
3	10.0178749274099	larger input
20	242582597.704895	upper boundary

Problem 6: Git and GitHub Commands Used

Repository

https://github.com/arvind-mocha/SOFTWARE_ENGINEERING_PROCESSESUMMER_2026

- `git init -b main` Initialised the project folder as a Git repository with `main` as the default branch.
- `git status` Displayed modified, untracked, and staged files before each commit.
- `git add .` Added all current project changes to Git's staging area.
- `git commit -m "message"` Created a permanent project revision with a descriptive commit message.
- `git remote add origin URL` Connected the local Git repository to the public GitHub repository.
- `git remote -v` Verified that the correct GitHub repository was configured as `origin`.
- `git push -u origin main` Uploaded the initial commit history and linked the local `main` branch to GitHub.
- `git push origin main` Uploaded subsequent local commits to the public GitHub repository.
- `git log --oneline` Displayed a concise list of commit identifiers and commit messages.

Problem 6: Version Control Evidence

High-quality commits

- Add validation for empty and non-numeric input
- Added real number support feature
- Added sinh calculation logic from scratch
- Added GUI using tkinter
- Pending features added to the GUI

Problem 6: Commit History

- 1 docs: add D2 README with setup and run instructions

Explanation: Added the project description, requirements, supported input range, repository structure, and instructions for running the application.

- 2 feat: implement from-scratch Maclaurin series for $\sinh$

Explanation: Added the core numerical algorithm for calculating $\sinh(x)$ without using prohibited Python mathematical-library functions.

- 3 feat: add Tkinter GUI with input validation and helpful errors

Explanation: Added the graphical interface, user-input controls, validation, exception handling, and understandable error messages.

- 4 refactor: separate parsing, calculation, and GUI responsibilities

Explanation: Separated input processing, mathematical calculation, and interface logic to improve readability and maintainability.

- 5 docs: update D2 requirements and traceability

Explanation: Documented the updated requirements and connected them to the from-scratch algorithm, GUI, exception handling, and repository changes.

Problem 7: Updated Requirements

ID	Requirement	Priority
REQ-F3-D2-01	The system shall provide a Tkinter graphical user interface.	High
REQ-F3-D2-03	The system shall support inputs in $[-20, 20]$.	High
REQ-F3-D2-04	The system shall calculate $\sinh(x)$ using a from-scratch Maclaurin series.	High
REQ-F3-D2-05	The system shall not use prohibited math-library functions for calculation.	High
REQ-F3-D2-07	The system shall provide helpful errors for invalid inputs.	High
REQ-F3-D2-12	The source code shall be placed in a public distributed version control system with a README.	High

Problem	Prompt type	How output was used
P5	Implementation-generation prompt with constraints	Helped structure scratch algorithm, GUI, and exceptions.
P6	Documentation and version-control planning prompt	Helped prepare README and commit-message examples.
P7	Requirements-revision prompt	Helped update requirements and trace D1 changes to D2.

For D2, the F3 calculator was modified from a D1 textual implementation into a Tkinter GUI application. The calculation now uses a from-scratch Maclaurin series, includes exception handling with helpful messages, has repository documentation, and includes an updated list of requirements aligned with the D2 constraints.